```
            "Could not find recipe with id {RecipeId}", id);    ❸
        return NotFound();
    }
    return Page();
}
```

❶ An ILogger instance is injected into the Razor Page using constructor injection.

❷ Writes an Information level log message

❸ Writes a Warning level log message

The `LogInformation` and `LogWarning` extension methods create log messages with a log level of `Information` and `Warning`, respectively. There are six log levels to choose among, ordered here from most to least serious:

- `Critical`—For disastrous failures that may leave the app unable to function correctly, such as out-of-memory exceptions or if the hard drive is out of disk space or the server is on fire.
- `Error`—For errors and exceptions that you can't handle gracefully, such as exceptions thrown when saving an edited entity in EF Core. The operation failed, but the app can continue to function for other requests and users.
- `Warning`—For when an unexpected or error condition arises that you can work around. You might log a `Warning` for handled exceptions or when an entity isn't found, as in listing 26.3.
- `Information`—For tracking normal application flow, such as logging when a user signs in or when they view a specific page in your app. Typically these log messages provide context when you need to understand the steps leading up to an error message.